

SOFTWARE ARCHITECTURE AND DESIGN SPECIFICATION

Online Exam Portal

Item	Details
Project	Online Exam Portal
Document	Software Architecture and Design Specification (SAD)
Version	1.0
Prepared by	Team 6: Chathurth R, Darshana S, Arati, Prakash N Heroor
Institution	PES University, RR Campus
Status	Draft
Date	02-10-2026

Revision History

Version	Date	Author	Change Summary
1.0	29-09-2026	Team 6	Initial SAD prepared from the Online Exam Portal SRS and supplied SAD template.

Approvals

Role	Name	Signature/Date
Project manager	CHATHURTH R	30-09-2026
Testers and documentation	DARSHANA S	30-09-2026
Developer	PRAKASH N HEROOR	30-09-2026
Developer	ARATI	30-09-2026

Table of Contents

- 1. Introduction
 - 1.1 Purpose
 - 1.2 Scope
 - 1.3 Audience
 - 1.4 Definitions
- 2. Document Overview
 - 2.1 How to Use This Document
 - 2.2 Related Documents
- 3. Architecture
 - 3.1 Architectural Goals and Constraints
 - 3.2 Stakeholders and Concerns
 - 3.3 Component Diagram
 - 3.4 Component Descriptions
 - 3.5 Chosen Architecture Pattern and Rationale
 - 3.6 Technology Stack and Data Stores
 - 3.7 Risks and Mitigations
 - 3.8 Traceability to Requirements
 - 3.9 Security Architecture
- 4. Design
 - 4.1 Design Overview
 - 4.2 UML Sequence Diagrams
 - 4.3 API Design
 - 4.4 Error Handling, Logging and Monitoring
 - 4.5 UX Design
 - 4.6 Open Issues and Next Steps
- 5. Appendices
 - 5.1 Glossary
 - 5.2 References
 - 5.3 Tools

1. Introduction

1.1 Purpose

This SAD defines the architecture, major components, design decisions, security controls, interfaces and interaction flows for the Online Exam Portal. It translates the requirements in the project SRS into an implementable architectural and design view.

1.2 Scope

The Online Exam Portal is a web-based system where instructors create multiple-choice exams and students take them within a timed window. The system grades responses automatically and shows results. Students can register, log in, view available exams, attempt an exam once, submit answers and view their results. Instructors manage their own questions and exams and view reports.

1.3 Audience

Developers, testers, course/capstone reviewers, instructors and maintenance teams. The section structure follows the supplied ATM SAD template.

1.4 Definitions

Term	Meaning
SAD	Software Architecture and Design Specification
SRS	Software Requirements Specification
MCQ	Multiple-Choice Question
Attempt	One student's single sitting of an exam
Exam Window	Time between exam start and end date-times in which it can be started
FR / NFR	Functional / Non-functional Requirement
CSRF	Cross-Site Request Forgery
ORM	Object-Relational Mapping
RBAC	Role-Based Access Control

2. Document Overview

2.1 How to Use This Document

Section 3 describes architecture, components, architectural pattern, technology choices, risks, traceability and security. Section 4 describes detailed design flows, UML sequences, API contracts, error handling, monitoring, UX and open issues. Section 5 lists glossary, references and tools.

2.2 Related Documents

Document	Purpose
Online Exam Portal SRS v1.0 Draft	Requirements, constraints, business rules, data fields and traceability.
Online Exam Portal Test Plan	Test cases linked to functional and non-functional requirements.
Course SAD template	Expected architecture and design structure.
Project GitHub repository	Source-code and documentation version control.

3. Architecture

3.1 Architectural Goals and Constraints

Area	Decision / Constraint
------	-----------------------

Modularity	Separate presentation, application/business logic and data access.
Security	Django sessions, password hashing, CSRF protection and server-side role checks.
Performance	Normal pages within 3 seconds; evaluation/result within 2 seconds for up to 50 questions; at least 10 concurrent students.
Technology	Python 3.11+, Django 5.x, HTML/CSS/JavaScript, SQLite.
Functional scope	Only MCQs with exactly four options and one correct answer in v1.0.
Timing	Server clock is authoritative for exam timing.
Deployment	HTTP during local development; HTTPS if deployed.

3.2 Stakeholders and Concerns

Stakeholder	Primary Concerns
Student	Ease of use, correct timing, secure exam and immediate results.
Instructor	Question/exam management, publishing and class reports.
Administrator	Account management through Django admin.
Developers	Modularity, maintainability and clear traceability.
Testers/Reviewers	Testability, predictable errors and security controls.

3.3 Component Diagram

The architecture is a layered Django web application. The browser is the client, Django provides presentation and application services, the ORM provides data access, and SQLite stores persistent data.

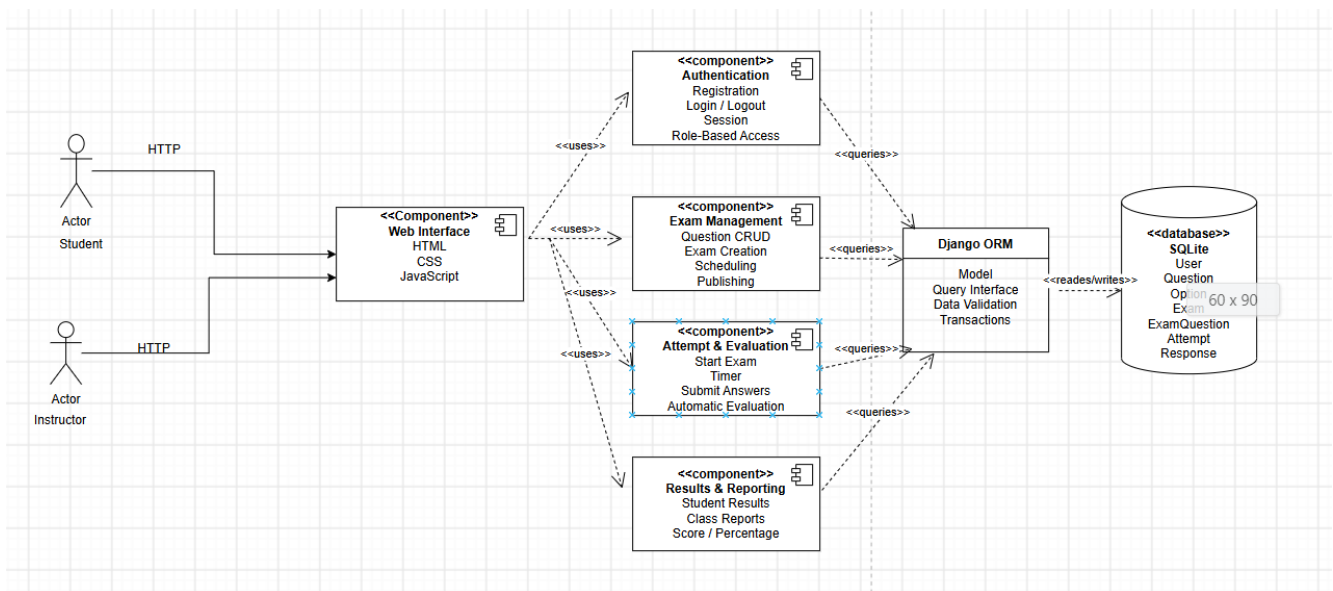


Figure 1. Online Exam Portal component/layered architecture.

3.4 Component Descriptions

Component	Responsibilities
Web Browser / UI	Registration, login, dashboards, exam instructions, exam attempt, timer/navigation, results and instructor forms/reports.
Django URL/View Layer	Routes requests, validates input, invokes application logic and returns HTML responses.

Authentication & Session	Registration, login/logout, session lifecycle, password hashing and protected-page checks.
Exam Management	Question CRUD, exam creation, scheduling, publishing and instructor ownership checks.
Attempt & Evaluation	Starts attempts, enforces one-attempt rule, handles deadlines, saves responses and calculates score.
Results & Reporting	Student result display and instructor class reports with access restrictions.
Django ORM / Models	Maps application objects to database records and performs database access.
SQLite	Stores User, Question, Option, Exam, ExamQuestion, Attempt and Response data.
Django Admin	Administrative account management; not a separate student/instructor feature.

3.5 Chosen Architecture Pattern and Rationale

A layered (three-tier) architecture is selected. The presentation layer handles browser-facing screens and forms; the application layer handles authentication, exam management, attempts/evaluation and results; the data-access layer uses Django ORM with SQLite. This matches the self-contained Django application described in the SRS and keeps the v1.0 implementation modular without the operational complexity of microservices.

3.6 Technology Stack and Data Stores

Layer	Technology / Data Store
Client	Chrome/Firefox; HTML, CSS, JavaScript
Application	Python 3.11+, Django 5.x
Authentication	Django sessions and password hashing
Data access	Django ORM
Database	SQLite
Communication	HTTP locally; HTTPS when deployed
Diagram/API tools	draw.io, PlantUML, Swagger/OpenAPI if API documentation is required

3.7 Risks and Mitigations

Risk	Mitigation
Network interruption	Server-side timing; persist submissions before showing result; retry if save fails before deadline.
Unauthorized access	Server-side role checks for every protected request.
Answer leakage	Correct answers are never sent to the browser during an active exam.
Data loss	Persist submitted attempts and back up SQLite before demonstrations.
Invalid question data	Validate non-empty text, four non-duplicate options and exactly one correct option.
Timing inconsistency	Use the server clock as authoritative.

3.8 Traceability to Requirements

Requirement	Architectural Component(s)	Design Evidence
FR-1 Registration	Authentication & Session	Validation, unique email, hashed password, account creation.
FR-2 Login/Logout/RBAC	Authentication & Session	Session lifecycle and server-side role checks.
FR-3 MCQ Management	Exam Management	Question/Option model and instructor ownership.
FR-4 Exam Scheduling	Exam Management	Exam window/duration validation and publishing.
FR-5 Attempt Exam	Attempt & Evaluation	Attempt creation, one-attempt rule

		and server timer.
FR-6 Submit & Evaluate	Attempt & Evaluation	Submission lock, score calculation and persistence.
FR-7 View Results	Results & Reporting	Student results and instructor reports.
NFR-1 Performance	All layers	Efficient requests and ORM queries.
NFR-2 Safety	Data/Evaluation	Persistent submission and protection of attempted data.
NFR-3 Security	Auth/Application/Data	Hashing, sessions, CSRF, RBAC and answer protection.
NFR-4 Usability	Presentation	Browser compatibility, responsive layout and consistent messages.

3.9 Security Architecture

The SRS specifies salted password hashes, server-side role checks, CSRF protection, Django ORM access, protected sessions and withholding correct answers during active exams. OWASP ASVS can be used as a supplementary security-verification checklist for web-application controls.

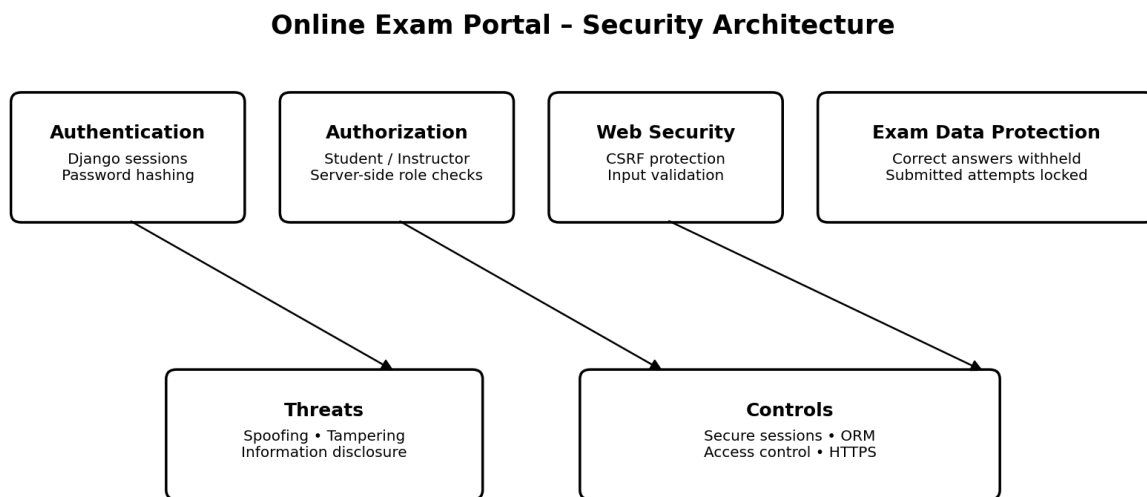


Figure 2. Security architecture and controls.

4. Design

4.1 Design Overview

The design uses the SRS entities User, Question, Option, Exam, ExamQuestion, Attempt and Response. An Instructor owns questions and exams. ExamQuestion maps selected questions to an exam. A Student creates an Attempt for an open exam, submits Responses and receives a stored score and percentage.

4.2 UML Sequence Diagrams

The SAD template requires at least two project-specific sequence diagrams. Two key flows are provided below.

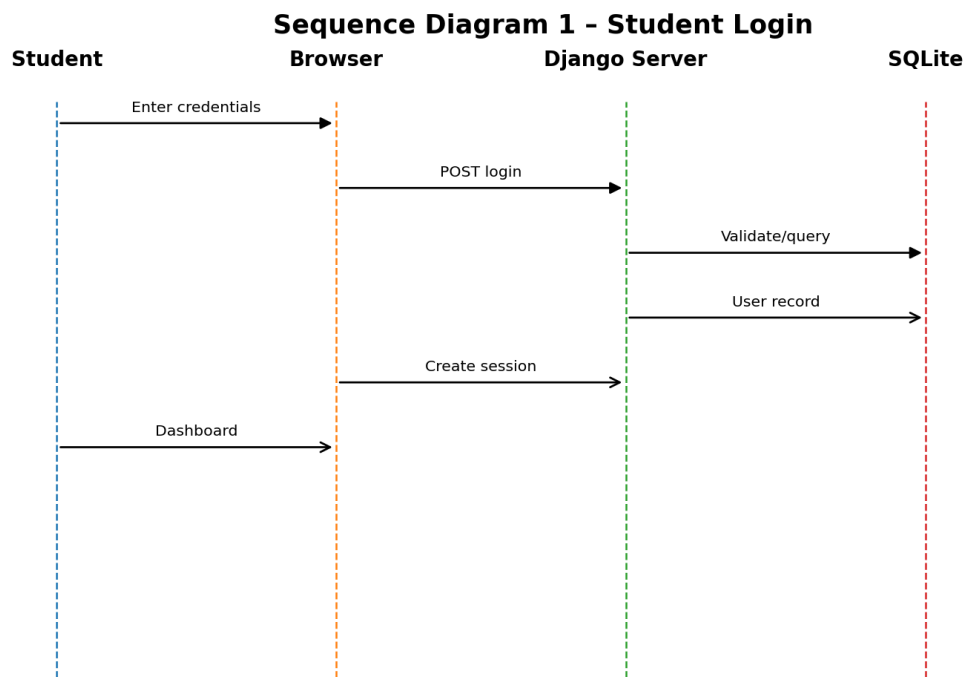


Figure 3. Student login sequence.

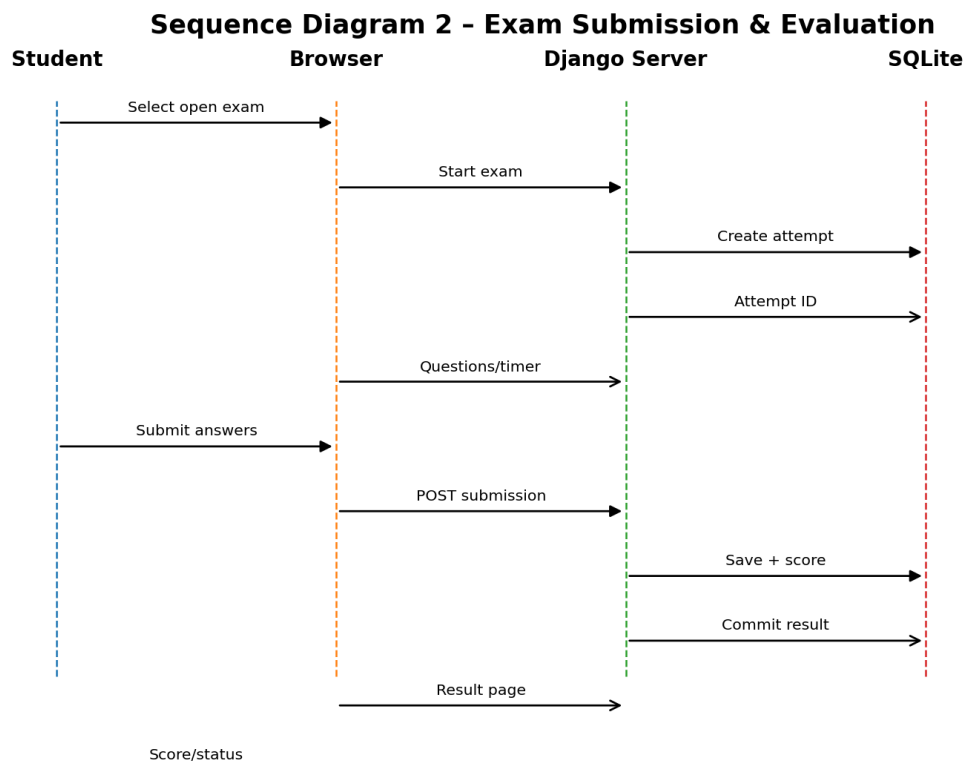


Figure 4. Exam submission and automatic evaluation sequence.

4.3 API Design

The SRS describes a Django server-rendered web application rather than a mandatory public REST API. The following HTTP interfaces capture the main request/response contracts for the system and can be implemented as Django views/forms or exposed as REST endpoints if required.

Endpoint	Method	Request	Response / Errors
/login	POST	email, password	Role dashboard; invalid credentials → authentication error.
/exam/{exam_id}/start	POST	authenticated student, exam_id	Attempt ID and timing data; closed exam/second attempt → 4xx.
/attempt/{attempt_id}/submit	POST	selected answers	Stored score, percentage and submission status; save/deadline failure → error.
/results/{attempt_id}	GET	authenticated student	Score, total, percentage and Pass/Fail; unauthorized → 403.
/instructor/exam/{exam_id}/report	GET	owning instructor	Student-wise report and summary; other instructor → 403.

If a formal REST API is introduced, its schemas should be documented with OpenAPI/Swagger and versioned.

4.4 Error Handling, Logging and Monitoring

Situation	Expected Handling
Invalid registration	Field-level message; account is not created.
Invalid login	Generic “Invalid email or password” message.
Unauthorized role access	403 access-denied response.
Exam not open	Reject start and show exam-status message.
Second attempt	Reject because each student has one attempt per exam.
Late submission	Server rejects answers after deadline.
Submission save failure	Show error and keep attempt open for retry before deadline where possible.
Unauthorized result/report	Reject access and do not reveal another user's data.

Operational logs may include authentication failures, exam-start failures, submission failures, evaluation duration and server errors. Passwords and sensitive credentials must never be logged. Monitoring should track response time, submission failure rate, evaluation duration and application availability. The SRS does not mandate a specific logging framework.

4.5 UX Design

Screens include Register, Login, Student Dashboard, Exam Instructions, Exam Attempt, Result, Instructor Dashboard, Question Form, Exam Form and Class Report. Each screen has the user's name and Logout action. Validation errors appear next to their fields. During an exam, one question is shown at a time with four options, Previous/Next buttons, a question navigator and a countdown timer. The SRS also requires Chrome/Firefox compatibility, usable layouts from 360 px upward and starting an available exam within three clicks of login.

4.6 Open Issues and Next Steps

Item	Status / Next Step
Formal REST API	Not mandatory in SRS; add OpenAPI documentation if introduced.
HTTPS deployment	Use HTTPS when deployed; localhost HTTP is for

	development.
Database scaling	SQLite is the v1.0 choice; reconsider only if deployment needs exceed it.
Exam enrollment	Per-student enrollment is out of scope for v1.0.
Question types	Only single-correct-answer MCQs are supported in v1.0.
Administration	Use Django admin for administrator account management.
Security review	Run an OWASP ASVS-based review before deployment.

5. Appendices

5.1 Glossary

Term	Meaning
Django ORM	Object-relational mapping layer used for database access.
RBAC	Role-Based Access Control for Student/Instructor permissions.
Session	Server-managed authenticated state for a logged-in browser.
Exam Attempt	One student's single sitting for an exam.
Exam Window	Time between configured start and end times.
CSRF	Cross-Site Request Forgery.
SAD	Software Architecture and Design Specification.

5.2 References

1. Online Exam Portal SRS v1.0 Draft, Team 6, PES University, 7 September 2026; revised 30 September 2026.
2. Supplied course Software Architecture and Design Specification template: ATM System example, Version 1.0.
3. ISO/IEC/IEEE 42010:2022, Software, systems and enterprise — Architecture description.
4. OWASP Application Security Verification Standard (ASVS), Version 5.0.0.
5. Django Documentation, <https://docs.djangoproject.com/>
6. NIST SP 800-160, Systems Security Engineering.

Reference note: ISO/IEC/IEEE 42010:2022 defines requirements for the structure and expression of architecture descriptions. OWASP ASVS provides security requirements for designing, developing and testing modern web applications.

5.3 Tools

Tool / Technology	Use
draw.io	Architecture and UML diagrams.
PlantUML	Optional text-based UML generation.
Swagger / OpenAPI	API contract documentation if REST endpoints are exposed.
Django	Web application framework.
SQLite	Database specified in the SRS.
Git / GitHub	Version control and project documentation.

End of Software Architecture and Design Specification